LLMCOMPASS · 建模专题调研 · 2026-10-29

点积脉动阵列的数据流，如何改变 L0A/B 与 C/D 的带宽成本

先确定数据在哪里驻留、哪些维度在空间上并行，再决定多少访问会跨越 SRAM 边界。精度配比、OS/WS 标签和峰值 MAC 数，都不足以单独决定 CACTI 的输入。

证据范围：本地 LLMCompass、SCALE-Sim 2.0.2 实际 demand 探针、官方 SCALE-Sim v3 / Gemmini / NVIDIA 文档、TPU 与 MFSA 原始文献。新增内容为研究报告及独立证据脚本；未修改现有模拟器实现。

---

1. 统一主评估模式是合理的，但还要固定累加边界

针对自定义点积单元，建议把 **WS + 明确的外部 FP32 result accumulator** 作为第一条研究主线，固定权重预装协议、输出更新规则及数组分区方式，再比较 BF16:FP8:FP4 = 1:2:4、1:2:6、1:2:8 和 1:4:8。OS 作为另一条独立对照线。这样可以区分收益来自新增算力、改变扩展方向，还是切换了数据流。

这里的“统一”是统一一组可检验的硬件与调度假设，不是把所有 NVIDIA / 昇腾芯片都解释为同一种微架构。外部 accumulator 可以由寄存器、专用片上存储或 SRAM 实现；不同实现必须绑定不同的访问及成本模型。

> **本次对旧模型的修正：**当前 `resident` 与 `l0c_rmw` 仅选择部分和的保存方式，不能分别等同于 OS 与 WS。旧模型始终把 PE 行列映射为 M/N，A/B 每个点积组都流入；真正 WS 的 PE 行通常映射到 K 的分组，M 随时间流动。只换一个累加开关无法表达这个变化。

术语也要固定：本文采用 B 是权重、A 是激活的约定，因此 WS 表示 B 驻留，A 驻留称 IS，OS 驻留输出部分和。如果你的矩阵写法把权重放在左侧 A，A 驻留也可以叫 WS；模型应显式记录 `stationary_operand=A/B/P`，不根据字母猜测。**WS 并非“不在 PE 中做加法”**：部分和可以沿 PE 链逐级累加，只是同一输出不必长时间驻留在同一个 PE。

---

2. C/D 是逻辑读写角色，不一定是两块 SRAM

本文统一用 **D = A × B + βC**，P 表示中间部分和。C 是旧值，D 是新值；二者可以是同一个 accumulator 地址的更新前后状态。Gemmini 文档的 C/D 命名与这里相反，阅读其接口时要按角色对应。最终 D 的格式也可以与 FP32 P 不同，量化及导出应另列事件。

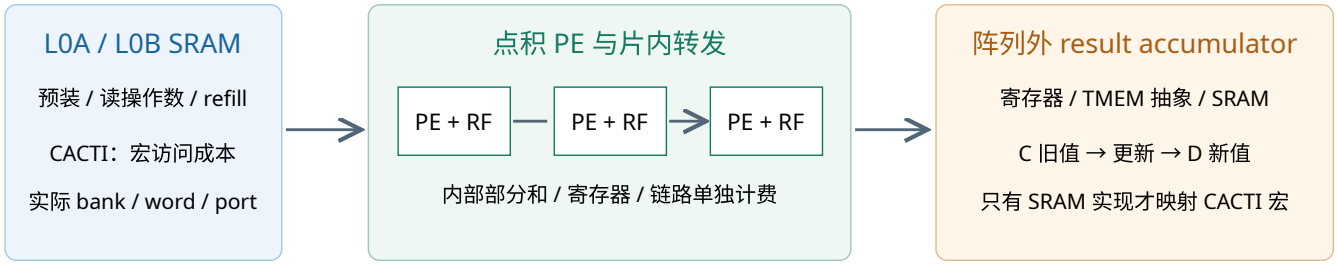


图 1｜三种动作必须分账。图为本报告的机制示意，不是任何厂商的物理布局。上一级 L1/HBM 搬运还在图外，不能和 L0→阵列供数混合统计。

至少要维护四类访问账本：PE 本地状态读写、PE 之间转发、阵列边界输入/输出、L0/ACC 的实际存储宏访问。驻留在 PE 寄存器中的部分和仍消耗寄存器与加法器能量；它只是没有因此产生阵列外 SRAM 事务。

原始实现证据 Gemmini 支持 OS/WS，WS 可把阵列输出送到带加法器的外部 accumulator SRAM。其 WS 通常让阵列的初值通路为零，在外部高精度累加器中合并结果。这说明“初值进入阵列”与“外部 ACC 读旧值”是两种不同实现，不能同时重复计数。[Gemmini]

点积 PE 证据 MFSA 预印本明确采用驻留权重的 dot-product PE、横向激活传递及纵向 FP32 部分和链，每列输出进入阵列外 result accumulator。其机制说明低精度并行度增加时，高精度部分和路径可以保持位宽不变；不代表用户的 1:2:6 设计无需增加输入链路。[MFSA §II-A，2026-10-14 预印本]

商用公开实例 TPU v1 原始论文描述 256×256 MAC 阵列每拍输出一个含 256 个部分和的向量，而不是 65,536 次外部 accumulator 写入。这是内部计算数量与边界输出数量不同的直接例子。[TPU，§2，第 3 页]

对 NVIDIA，需要另设指令层契约。官方资料将 WGMMA 的累加器放在寄存器、tcgen05 的累加器放在 TMEM；这些位置本身不能反推出全部内部 PE 数据流。PTX 确实还定义了具名 `tcgen05.mma.ws`，应按该指令及其目标架构支持解释，而不能把所有 tcgen05 泛称 WS。公开文档也不足以还原 TMEM 的 SRAM 宏结构。[WGMMA] [tcgen05] [PTX WS 指令]

---

3. 同样的 R×C 个点积 PE，在 OS / WS 中映射不同

定义物理 PE 行列为 R、C，每个 PE 一次发射归约 L 个 K 元素，发射间隔为 II 个阵列周期。计算峰值为 R×C×L/II MAC/cycle；1 MAC 按一个乘累加计。H16Q32E64 表示 L 分别为 16、32、64，不能把算法完整 K 与 L 混用。



图 2｜箭头及网格为示意，不代表真实延迟。WS 增加物理行数是在扩大 K 覆盖范围；不能沿用 OS 的“行数增加就是 M 并行增加”。

OS：输入持续流入，输出在驻留区间结束时离开

设一个完整 R×C 输出 tile 的 K 全程驻留在 PE 中，无中途 spill。A 在行内复用、B 在列内复用。满发射时的稳态平均需求如下，单位 Byte/cycle；单次发射的 burst 不除以 II：

```
A_OS = R × L × a_bits / (8 × II)
B_OS = C × L × b_bits / (8 × II)
Q = ceil(K / L)
PE partial-sum state = R × C × p_bits / 8
输出 tile 最终总写量 = R × C × d_bits / 8
仅按计算窗口归一化的最终输出均值 = R × C × d_bits / (8 × Q × II)
```

“最终只写一次”描述的是每个输出元素的总访问次数，不是带宽恒定。K 扩展使 Q 减小，完成 tile 更快，最终输出的窗口平均带宽也会上升。若 K 很长，输出均值可能远低于 A/B；若 K 很短或 MN 并行度很大，输出排空仍可能成为瓶颈。

OS 不要求把整个 R×C tile 在同一拍写到 SRAM。可以每拍导出一行、经 FIFO 排空，或双份 PE 累加状态交替计算与导出。旧模型把整个 tile 作为一次 writeback burst，是一种接口假设，不是 OS 的必然需求。反过来，只有两份 L0 缓冲也不能证明 PE 计算与结果排空能够重叠。

WS：B 预装，A 持续流入，部分和经过 PE 链

采用图 2 的典型 WS：每个 PE 持有 L 个权重，R 行覆盖最多 R×L 个 K 元素，C 列对应 C 个 N 输出。先装 B，再流过不同的 M 行。对一个满映射权重 tile：

```
K_span = R × L
F = ceil(K / K_span)
A_WS_stream = R × L × a_bits / (8 × II)
B_WS_current_tile_stream = 0      # 当前权重已驻留；下一 tile 预装另算
B_preload_tile_bytes = R × C × L × b_bits / 8
阵列边界结果速率 ≤ C × p_bits / (8 × II)
```

“B 的计算阶段读取为零”不表示权重免费。预装、跨 tile 重装、下一组权重的后台装载以及 PE 权重寄存器的面积和能耗，都必须保留。能否用较窄 L0B 端口，还取决于预装截止时间、PE 装载网络及是否有双份权重状态。

若每个 K fold 的边界输出都在同一 SRAM accumulator 中物化，β=0、第一次写可省略读零值，p_bits=d_bits，且最终 D 与该存储原地重合，则整个 GEMM：

```
ACC_write_bytes = F × M × N × p_bits / 8
ACC_read_bytes  = (F − 1) × M × N × p_bits / 8
最终结果字节 = M × N × d_bits / 8
```

上述 ACC_write 已包含最终 D 的存储，不能再把最终结果字节加一次。若最终量化、p_bits≠d_bits 或 D 使用独立存储，要显式增加 epilogue 的读写。这是一个明确的存储协议，而不是 WS 定义本身。若 β=1 且初值也从该 ACC 读取，读量改为 F×M×N×p_bits/8；若 C 初值来自独立缓冲，则单独统计该通道。若中间结果驻留在另一份寄存器状态中，L0C 可以只承担最后一次存储，但不能漏算那份寄存器的更新和容量。

当 F>1 且跨 fold 保留状态时，WS 在 N-tile → K-tile → M-stream 顺序下跨所有 M 复用 B，需要保留约 M×C 个外部部分和。F=1 时可以直接流出最终 D，不一定需要保留整个 M×C 块。若所需状态的容量只容纳 M_tile×C，调度必须再次分块，权重可能重装。**保留更多 B 与保留更多 C/D 会竞争有限片上容量。**

3.1 驻留与多周期加载：什么时候 SRAM lane 可以不增加

“SRAM → 寄存器/局部 RF → 阵列”的层次设计能够把 SRAM 的装载带宽与 PE 的内部消费带宽分开。这是有效的研究假设。对用户提出的 A 驻留场景，需要描述 A tile 在寄存器中保留多久、下一 tile 何时必须准备好，以及装载能否与当前计算并行。

```
V_A = 下一份驻留 A tile 实际需要装入的物理字节
W_A = 可用于预装这份 tile 的周期窗口
T_load_A ≈ startup + ceil(V_A / BW_A_effective)
保留现有 A SRAM lane 的必要服务条件：T_load_A ≤ W_A
还需满足：RF 容量、装载端口、bank 冲突及计算/装载状态隔离
```

例如原 A tile 为 8 KiB，L0A 有效带宽 128 Byte/cycle，忽略启动延迟时需 64 周期。K 扩展使 tile 变为 16 KiB，需要 128 周期。如果每份 A 都可复用 256 周期，原 SRAM lane 仍可能够用；如果可用窗口只有 64 周期，则原带宽不够。更长点积可能增加 V_A，也可能使重装间隔缩短，不能仅用“它被复用”断言 SRAM 永不需要扩宽。

若只有一份 A 驻留寄存器，不能在计算仍使用它时任意覆盖成下一份 A。完整重叠通常需要双份寄存器状态、可证明安全的分区更新，或其它装载协议。L0 的双缓冲不自动等于 PE 权重寄存器的双缓冲。即便 SRAM 面积增量为零，额外 RF 容量、读端口和分发线仍可能增加面积与功耗。第一份 tile 的启动装载也不会被此前不存在的计算隐藏。

对于同一 A 驻留映射，B 是流动操作数。如果 B 每拍需要新的数据，且更近层没有复用或缓存命中，L0B 的长期供给必须跟上实际消费；队列只能平滑短期突发，不能消除长期带宽缺口。C/D 是否具有同样压力取决于边界输出频率，以及它们落在 RF、专用存储还是 SRAM。RF/TMEM 端口增加不等于 L0 SRAM lane 必须同步增加。

“频繁切换数据”还应具体化为每周期多少个不同物理 word 事务。数据值的电气翻转率影响动态能耗，访问次数与并行服务需求决定端口压力；二者不能混为 SRAM lane 数量。

OS 可以多周期写出，但“可忽略”必须是通过约束后的结论

```
V_D = 一个输出 tile 的物理写回字节
W_D = 在结果缓冲耗尽或覆盖前可用的排空窗口
T_drain_D ≈ startup + ceil(V_D / BW_D_effective)
保持 C/D SRAM lane 不变：T_drain_D ≤ W_D，且长期产出 ≤ 长期服务
还需满足：结果队列容量、bank/端口、状态切换和依赖约束
```

16×16 FP32 输出是 1024 Byte；已有输出通道为 64 Byte/cycle，则需 16 个理想传输周期。K=4096、L 从 64 增到 128，计算窗口从 64 缩到 32 周期：如果有独立结果暂存并允许重叠，原输出通道仍够用，可以让 C/D 的 **SRAM 面积增量保持零**。但 K=512、L=128 时，计算窗口只有 4 周期，连续输出的均值为 256 Byte/cycle；原 64 Byte/cycle 无法无限期维持，有限 FIFO 最终会满。

因此，“忽略 OS 的 C/D 扩展”应改为“确认原端口满足排空和依赖约束后，令该设计点的 C/D lane 增量为零”。最终访问能量、已有容量及 PE 累加器成本仍然存在。如果没有计算/排空重叠能力，则需把 drain 加入延迟，不能用较窄 SRAM 的结果同时宣称原峰值吞吐。

已有研究如何支持这个方法

一手研究 / 工具	与本问题有关的机制	对本模型的启示
Eyeriss，ISCA 2016	区分本地存储复用、PE 间通信和更远存储访问；在共同硬件约束下分析数据移动	复用收益要落实到具体存储层；RF 的成本不能隐藏在“免费复用”中
MAESTRO 官方模型	根据映射估计缓冲需求；NoC/片外带宽约束会改变运行时间	把容量、带宽和运行时间联合评估；不足带宽应表现为性能限制
Timeloop/Accelergy 约束	显式描述时间/空间映射、keep/bypass、复用及首次 RMW 更新	“哪份数据保留在哪层、何时真正读写”应是输入契约，而非 OS/WS 的隐含常数
CUTLASS Efficient GEMM	寄存器中分别保存当前计算 fragment 和下一次取回的 fragment；epilogue 独立组织结果写出	多周期搬运与计算重叠有具体的状态成本，不能只增加一个“预取”标签

这些工作支持“分层复用 → 实际访问 → 资源约束 → 成本”的方法论；它们不证明所有 OS 都可忽略输出端，也不自动给出你的 12nm/22nm 宏 PPA。对固定 SRAM 容量，可以由满足目标吞吐/延迟的候选 bank 数、字宽和端口组织调用 CACTI 估算宏面积，再加独立 RF、FIFO 与互连成本。**这是合理的架构级预测方法，前提是服务窗口和物理组织可实现。**

---

4. 精度配比相同，K 扩展与 MN 扩展也不是同一种硬件

1:2:8 只规定各精度的相对峰值，没有规定增加的 FP4 MAC 用于更长点积、更多同时输出，还是更多独立阵列。对 FP4 从 1:2:4 增到 1:2:8，新增倍率 s=2；增到 1:2:6，s=1.5。FP8 从 1:2:4 到 1:4:8 的新增倍率也是 2。比较同一精度的设计时位宽不变，不能再把新增算力用一次“位宽减半”抵消。

先在一个明确的矩阵更新粒度上比较

假定一次可见更新计算 mμ×nμ×kμ，输入没有被更近的缓冲命中，C/D 在这个粒度读写一次。若 mμ、nμ、kμ 分别扩为 u、v、w 倍，则：

```
MAC/update ∝ u × v × w
A payload/update ∝ u × w
B payload/update ∝ v × w
C/D payload/update ∝ u × v
独立 accumulator 数量 ∝ u × v
```

因此 K 扩展优先增加输入供数；MN 扩展还增加同一更新粒度的 C/D 状态和数据量。但这是**每次更新的 payload**，不能直接写成每拍带宽。必须除以该更新的实际 II，并应用驻留、collector 命中、转发、压缩与存储事务规则。NVIDIA MMA 的指令 m/n/k 也不能直接当作私有 PE 阵列的 R/C/L。

固定精度增加 s 倍 MAC	A / B 每更新数据量	C/D 每更新数据量	额外状态
kμ × s	A × s；B × s	×1	输出状态不增；归约硬件改变
mμ × s	A × s；B ×1	×s	输出状态 ×s
nμ × s	A ×1；B ×s	×s	输出状态 ×s
mμ ×u、nμ ×v，uv=s	A ×u；B ×v	×s	输出状态 ×s；广播网络改变

然后应用真正的 WS 驻留与空间映射

WS 实现选择	流阶段 A / 边界 P	B 预装与状态	跨 K 更新次数
L × s：PE 内点积加长	A ×s；P 向量宽度不变	满权重 tile 容量 ×s	F 可减少，呈取整台阶
R × s：更多 K 归约行	A ×s；P 向量宽度不变	满权重 tile 容量 ×s；链更长	F 可减少；填排延迟也变
C × s：更多 N 列	A 不变；P 向量宽度 ×s	满权重 tile 容量 ×s	F 不变
s 个 M 并行分区	A 合计 ×s；P 合计 ×s	驻留权重复制；源 SRAM 读取取决于 multicast	每个输出的 F 不变

这里的放大关系要求维度足够大、映射能填满、II 不变。特别是**WS 的 R 扩展属于 K 扩展**。真正增加 M 并行度需要多个独立 M 流及其部分和路径，可能通过阵列复制或可分区结构实现，不能只把已有 WS 阵列的 R 字段改大并称为 M 扩展。

这也回答了“N 很大时 C/D 压力更大”的两个含义：固定硬件时，N 更大主要增加 N tile 数、总工作量及总输出量，不自动增大瞬时端口宽度；若借此把硬件的 N 并行宽度做大，则 C/D 的并行更新宽度和存储需求确实增加。算法 shape、指令 tile shape、物理阵列 shape 必须分别记录。

> **不要把 OS 的 C/D 视为零成本。**OS 可以把多次 K 更新留在 PE 内，使外部 C/D 总量很小；在完整 GEMM 输出总字节不变时，加速仍使输出平均速率提高。与此相对，WS 的 K 扩展可能减少跨 K fold 的外部 RMW 次数。两者都需要“总量 + 窗口 + 峰值 + 实际端口调度”，不能只看其中一个。

---

5. 定量实例：同样翻倍算力，压力落在不同通道

本报告独立推导 固定 M=N=256、K=4096，FP4 A/B，FP32 累加，β=0、II=1。基线 R=C=16、L=64。WS 把每个 K fold 的边界结果写入外部 SRAM accumulator，第一次更新不读旧值。全部数字为理想计算发射窗口与有效 payload，未计预装、填排、SRAM word 补齐和 stall。

WS 设计	A 稳态输入 / 边界 P 输出 Byte/cycle	ACC 总读 / 总写	F / 理想发射周期
16×16×64 基线	512 / 64	0.75 / 1 MiB	4 / 16384
K 扩展：16×16×128	1024 / 64	0.25 / 0.5 MiB	2 / 8192
N 扩展：16×32×64	512 / 128	0.75 / 1 MiB	4 / 8192
M 扩展：两个基线 WS 分区	1024 / 128（合计）	0.75 / 1 MiB（合计）	每输出 4 / 8192

K 扩展减少了 ACC 的总读写，N/M 并行扩展则把相同 ACC 事务压到更短时间内。后两种方案可能要求更多 ACC 并行端口或 bank。K 扩展同时加重 A 的流带宽及单个权重 tile 的预装量/期限；本例整个 GEMM 的 B 预装总量仍为 K×N×b_bits/8，并未翻倍。因此不能只用 SRAM 面积或单个带宽数决定优劣。表中 M 扩展是两个各含 16×16 PE 的完整 WS 分区，合计 512 个 PE；它不是把原来的 256 个 PE 等分。相同峰值增长不代表相同面积，需接入各设计的 PPA。

同一 GEMM 若改用无 spill 的 OS，基线与三种翻倍设计的最终 FP32 输出都只有 256 KiB。按理想计算窗口均摊，输出从 16 增到 32 Byte/cycle。这个长 K 例子中 A/B 是主要供数压力，但实际结果排空速率仍取决于端口和是否能重叠；256 KiB 是总量，不是端口宽度。

更小的可比实例：权重复用与输出流不一定相互放大

M=64、N=16、K=1024、R=C=16、L=64 时，OS 与 WS 都需要 64 个理想计算 issue。OS 对 A/B 各读 32 KiB；WS 对 A 读 32 KiB，B 预装只需 8 KiB；两者最终 D 都为 4 KiB。这时 WS 的 K fold 只有一个，没有额外跨 K 的旧值读取。假设 B 装载带宽 512 Byte/cycle，8 KiB 预装至少需要 16 个传输周期；这些周期没有包括在前述 64 个计算 issue 中。

短 K 会破坏“配比提高就同比加速”的直觉

M=N=K=256，仍为 16×16，WS 不做独立分区时，H16/Q32/E64 只分别需要 16/8/4 个有效 K 行。低精度使数组更大比例闲置，三种精度都需要 4096 个理想计算 issue，计算槽利用率为 100%/50%/25%。OS 对应 4096/2048/1024 个 issue，因为其行仍映射 M。

若把 E64 的 16 行分为四个 4 行 WS 子阵列，分别处理四条 M 流，可以恢复更多算力利用率；但需要四个独立的部分和起点、结果出口及对应 accumulator 通道，还需要复制或广播权重。理想输出从 16 增到 64 个 FP32 值/拍，即 64→256 Byte/cycle。**通过分区释放的算力，必须由新增输出服务能力兑现。**

---

6. 交互推导：改变 shape 与点积长度

下面是公式计算器，不调用 CACTI 或 SCALE-Sim。OS 固定为无 spill、每次只保留 R×C 输出；WS 固定为 N-tile→K-tile→M-stream，外部 ACC 每个 fold 更新一次；WS 无独立分区。全程 β=0、FP32 P/D、II=1。有效字节不含 padding、元数据和 SRAM refill。

指标（有效 payload）	OS：PE 保留全 K	WS：每 fold 更新外部 ACC
理想计算 issue	16,384	16,384
计算槽利用率	100%	100%
A 入口有效总字节	8,192 KiB	8,192 KiB
B 入口有效总字节	8,192 KiB	512 KiB，全部为预装
A / B 流阶段均值，B/issue	512 / 512	512 / 0
ACC 总读 / 总写	0 KiB / 256 KiB	768 KiB / 1,024 KiB
最终 D 总字节	256 KiB	256 KiB
最终或部分结果的窗口均值，B/issue	16	64
WS K fold / 每 fold 覆盖	—	4 / 1024

B 预装列只计总量。平均输入按计算 issue 窗口归一化，不能替代真实端口逐拍峰值。最终 D 与 ACC 原地重合；导出到更远存储的额外读取/写入需另计。

---

7. SCALE-Sim 提供了什么，现有连接缺少什么

本地代码事实 本次本地运行的是 SCALE-Sim **2.0.2**。其三个生成器把 GEMM 操作数变成逐行的需求地址矩阵，空位用 −1 表示。OS 的空间行/列为 M/N，时间为 K；WS 为 K/N，时间为 M；IS 为 K/M，时间为 N。生成器利用 fold、skew 和 padding 表示不同数据流，并不是数值 MAC 仿真。

用真实生成器执行 2×2 标量 PE 阵列、M=N=K=4，统计非 −1 地址请求，得到：

数据流	A / B read 请求数	ofmap write 请求数	相同输出地址重复次数
OS	32 / 32	16	1
WS	32 / 16	32	2 个 K fold
IS	16 / 32	32	2 个 K fold

当 K 改为 8，OS 输出请求仍为 16，WS/IS 变为 64。这验证了当前生成器中“驻留输入复用”与“跨 K fold 重复输出”的关系。但 **ofmap 是 write buffer，没有另一个 C-read 通道**；重复写同一输出地址不等价于已经完成 SRAM accumulator 的 read-modify-write、RAW 依赖与加法器时序建模。

本地探针还发现 OS 的库计数器 `get_ofmap_requests()` 在这个例子返回 32，而实际 trace 只有 16 个非空写请求。探针保留了两种统计。因此不应直接把报告字段乘 FP32 字节数后当成物理 ACC 事务。

LLMCompass 在 `software_model/matmul.py` 中仅消费 SCALE-Sim 的总周期与整体利用率，之后按 `mac_per_clock` 缩放周期；没有同时重建宽点积操作数 trace。局部本地路径还固定 `word_size=1`，不能表达 A/B 的 BF16/FP8/FP4 与 FP32 P 的不同字节数。

SCALE-Sim v3 增加了布局、存储系统及 Accelergy action count 等能力，可借鉴其接口与执行框架；但每 PE 的 psum 寄存器/存储动作与阵列外 ofmap SRAM 动作仍需分开解释。升级版本本身不证明已经支持自定义点积归约、C/D 的存储协议及真实 accumulator 依赖。[SCALE-Sim v3 §VII]

真实 demand 探针：probe.py、probe.json。单位为逻辑元素请求 / nominal demand row，不是经过存储服务后的 Byte/cycle。没有在该探针中运行 DRAM、dot-product PE 或数值仿真。

---

8. 从数据流到 CACTI：必须加入阶段、布局与服务时间

新的模型链应是 **GEMM 与精度 → 空间映射/循环顺序 → 驻留与事件时间表 → 存储布局与端口服务 → CACTI 成本 → stall 反馈**。CACTI 仍负责宏的面积、单次访问能量和时序；它不决定某个 PE 使用 OS 还是 WS。

阶段或资源	OS 需记录	WS 需记录
预装	已有 C 初值、输入预取	B 预装、权重驻留范围与下一组预装截止时间
计算流	A/B 入口，PE 内 P 更新	A 入口，PE 间 P 转发，边界结果输出
跨 K	继续保留或 spill/reload	外部 ACC 更新/转发/再入阵列
结果导出	串行 drain、FIFO 或双份 ACC 状态	最后 fold 的结果导出及与其它 ACC 访问的端口竞争
上一级 refill	实际命中/重读，不默认每个 dot 都重填	源 SRAM 读与 PE 权重复制/广播分开计数

每个服务事件至少要包含时间或发射约束、操作数、地址/有效字节、源/目的层次、阶段、读写类型和依赖 ID。再把事件映射成 `(bank, row, word, byte_mask)`，按真实布局合并能合并的请求。多个 PE 共享同一 word 只读一次；不同地址不能因总字节恰好够一个 word 就虚构合并。

对一个候选 SRAM 组织，必须同时检查宏容量、每个 bank 的瞬时端口冲突、可持续 cycle time、读后写反馈延迟、FIFO 深度以及 preload/drain 的期限。可以缓存平滑突发，但缓存必须有容量和成本。仅用平均 Byte/cycle 或全阵列最大带宽乘“效率系数”，会遗漏阶段冲突和某个 bank 的热点。

```
E_dynamic = Σ_bank (served_reads × E_read + served_writes × E_write)
E_leakage = all_deployed_macro_leakage × actual_elapsed_time
E_total = E_SRAM + E_PE_local_state + E_dot + E_accumulator_add +
E_interconnect
Area_total = Area_TensorCore + Area_L0 + Area_external_ACC + Area_glue
```

成本边界要和你的综合数据对齐：若 Tensor Core 面积已经包含 PE 权重寄存器、部分和寄存器或 ACC 加法器，不能再加一次。只有确认由 SRAM 宏实现的组件才直接使用 CACTI；寄存器/TMEM 可先作为独立的已校准成本表。22nm CACTI 预测与 12nm 综合结果不能直接相加。

同一芯片若同时支持多个精度和 OS/WS，所有模式必须验证同一套物理端口和路由。已有 lane 能满足带宽和 deadline 时面积可以保持不变，访问频次仍会改变能量；只有超出服务能力后才搜索更多 bank、更宽 word 或更多端口。选择目标应包含延迟/能量，不应把不可服务的请求量对应的功率当成已实现功率。

8.1 昇腾与 NVIDIA：公开带宽是多少，能否供满计算

本节所有速率均先归一化为 **Byte/cycle，每个 AIC 或每个 SM**。不能把全芯片 HBM TB/s 与一个 SM/AIC 的本地需求直接比较。若所在时钟域为 f GHz，则 B Byte/cycle 对应 B×f GB/s；例如 128 Byte/cycle、1.8 GHz 对应 230.4 GB/s。这里的频率只是换算示例，设备实际频率应实测记录。

公开接口：L0 的写入、计算读取与结果排出分别建模

架构 / 单元	通路	Byte/cycle	证据口径
昇腾 2201，A2/A3，每 AIC	L1→L0A / L1→L0B	256 / 128	官方 PIPE_MTE1 搬运规格
昇腾 3510，每 AIC	L1→L0A / L1→L0B	256 / 256	官方 PIPE_MTE1 搬运规格
昇腾 3510，每 AIC	L0C→UB；L0C→L1	各 128	官方 PIPE_FIX 通路；不据此假定能同时合计 256
昇腾 Cube 内部接口	L0A/B→Cube、Cube↔L0C	未核实到独立峰值	上述搬运规格不能代替计算端读写规格
H100 / B200，每 SM	SMEM 读取	128，参考峰值	官方 bank 规则及研究微基准；无冲突、充足并行度
B200 / B300，每 SM	TMEM↔MMA、TMEM↔寄存器、SMEM→TMEM	分别待标定	公开容量 / 指令布局不足以给出统一吞吐值

昇腾 2201 文档对应 A2/A3 产品，L0A/B 各 64 KiB、L0C 128 KiB；3510 文档为 L0A/B 各 64 KiB、L0C 256 KiB。950PR/DT 的 MMAD 文档列出后一组容量。具体 910B/910C/950 SKU 应与编译目标、平台查询对应，不能只按商品名复用全部时序参数。[2201 官方规格] [3510 官方规格] [MMAD 产品约束]

NVIDIA 官方通用 shared-memory 规则为 32 个 bank、每 bank 每拍 32 bit，得到 128 Byte/cycle 的无冲突服务参考；FlashAttention-4 对 H100/B200 的 shared-memory 读吞吐采用这一数值，同时给出 BF16 每 SM 的 4096/8192 FLOP/cycle 参考。该数字不是为每个 Tensor Core 各提供 128 Byte/cycle，也不证明读写、TMA、MMA 所有通路可无冲突地同时满速。B300 可先用此通用 bank 参考作敏感性分析，再校准其实际 MMA 路径。[CUDA bank 规则] [FlashAttention-4]

B200 和 Blackwell Ultra 的 TMEM 为每 SM 256 KiB；H100 的矩阵累加器使用寄存器路径，没有同样的 tcgen05 TMEM。TMEM 的 128 lanes×512 columns×4 Byte 是容量组织，不能由此直接得到 512 Byte/cycle。`tcgen05.ld/st/cp`、MMA 累加更新和 A 操作数读取使用不同路径，不应套用一个未经校准的 TB/s。[PTX 官方语义] [Blackwell Ultra 官方说明]

昇腾供数预算：宽 L0 与较窄的预装通路并不矛盾

使用 16×16×16 FP16 每拍的参考吞吐，即 P=8192 FLOP/cycle。3510 官方文档明确给出这一计算能力；这里是理想稳态预算，不把单个 MMAD 的实际延迟当成一拍。若每次都为这个微块提供全新的 A/B，每路入口 payload 为 512 Byte/issue；若每个微块都在外部物化 FP32 部分和，则每方向 C/D payload 为 1024 Byte/issue。这些是**条件化的逻辑需求**，不是已测得的 SRAM 端口峰值，驻留和转发会改变实际访问量。

改看一个完整 128×128×64 FP16 tile：A/B 各 16 KiB，理想计算 256 周期。2201 的 L1→L0A 服务需 64 周期、L1→L0B 需 128 周期；按两次装载串行的保守预算为 192 周期，仍小于 256。3510 对应 64+64=128 周期。这说明在布局、重叠和依赖允许时，较窄的 MTE1 预装通路有机会供满计算，而不是要求上一级每拍重新传入 512+512 Byte。

同一 tile 的 FP32 输出为 64 KiB。3510 的 128 Byte/cycle 输出通路需 512 周期：若完整 K 只有 64，最终输出均值为 256 Byte/cycle，可能成为持续吞吐瓶颈；完整 K=256 时为 64 Byte/cycle，K=4096 时为 4 Byte/cycle。这里假定全 K 的结果一直保留、只有最后导出一次，并且输出可与其它计算重叠。中途 spill、量化及共享 FixPipe 竞争会改变预算。

NVIDIA 供数预算：更快的低精度算力会迅速吃完 SMEM 余量

对一次 m×n×k 更新，假设两操作数从 SMEM 各读一次、没有更近层复用，其理想读取需求为：

```
T_compute = 2 × m × n × k / P
BW_AB_required = (m × k × a_bytes + k × n × b_bytes) / T_compute
               = P × (m × a_bytes + n × b_bytes) / (2 × m × n)
```

示例 / 纯矩阵 payload	P，FLOP/cycle/SM	SMEM 读需求	相对 128 Byte/cycle 预算
H100 BF16，64×128×64	4096	96 Byte/cycle	75%
B200 BF16，128×128×64	8192	128 Byte/cycle	100%
B200 FP8，128×128×64	16384，归一化 2×	128 Byte/cycle	100%
FP4 基准 1:2:4，128×128×64	32768，归一化 4×	128 Byte/cycle	100%
自定义 FP4 1:2:6，相同读取次数	49152	192 Byte/cycle	若无额外复用，计算上限约 2/3
自定义 FP4 1:2:8，相同读取次数	65536	256 Byte/cycle	若无额外复用，计算上限约 1/2

以上忽略量化 scale、元数据、其它 SMEM 读写和 bank 冲突，不能当作实际 kernel 利用率。结果说明 1:2:4 的算力与数据位宽可相互平衡；单独提高某精度的算力时，SMEM 或更近接口必须承担更多供数，或减少从该边界读取的次数。把 A 搬入 TMEM 只有在跨更新复用、从已有 TMEM 结果直接消费或改变争用路径时才有收益；每次先从 SMEM 读一次再搬过去，不会自动消除源端流量。

TMEM 可先计算"所需速率"，同时将"可供速率"留待测量。例如 128×128×64 BF16 在上述 B200 参考下需 256 个理想周期，FP32 C/D 各 64 KiB；若该更新对 TMEM 各读写一次，其等效需求为每方向 256 Byte/cycle。保持更新形状、将算力提高，会提高此要求；若累加保留更久，则导出频率会下降。不能将这个等效需求当作物理 TMEM 事务或峰值端口规格。

最小实测：把未知端口补成芯片 profile

昇腾先扫 MMAD 的 M/N/K 与独立/交替 L0 地址，结合 `MemoryL0.csv` 中的 A/B 读写、`aic_l0c_read_bw_cube` 和 `aic_l0c_write_bw_cube`，以及 MTE1/FixPipe 活跃时间。分别测 L1→L0、L0 已就绪的重复 MMAD、跨 K 累加、最终输出，并报告每个 AIC 的实际周期与频率。计数器的算子均值不自动等于可持续峰值。[官方性能数据字段]

NVIDIA 分别测无冲突 LDS/STS、SMEM→MMA、TMEM→寄存器、寄存器→TMEM、SMEM→TMEM，再测这些路径与 MMA 更新的并发。H100/B200/B300 分别记录实际 SM 数、时钟、tile、指令、活跃 warp 与字节计数。优先把单条路径的 B/cycle 写入 profile，最后才按实际时钟换成 GB/s。

可复算文件：device_feed_budget.py、device_feed_budget.json。已公开参数、未知峰值与分析假设分别记录；本节没有运行实机微基准，也没有将需求推导冒充芯片内部规格。

8.2 真正用于 lane 设计的是 SRAM 端口访问轨迹

> **重要边界：**上节 128 Byte/cycle 及各供数表属于有效 payload 的吞吐预算。它们不是 B200 或昇腾的逐拍 SRAM 请求轨迹，不能直接据此断言实际宏字宽、bank 数或每拍读取次数。等效 128×2×16 只描述峰值计算量；实际端口时序必须由微架构实现、可靠仿真或实机访问证据约束。

模型要显式包含 SRAM 端口、装载及地址发生器、bank/字映射、operand latch/RF/collector、广播与分发、dot pipeline、部分和状态和写回控制。只有确实跨越目标 SRAM 边界的动作才成为该 SRAM 的访问；计算逻辑重复使用驻留值时，不应按 MAC 次数重新生成 SRAM 读请求。

每拍至少记录 `memory, cycle, bank, word_address, read/write, byte_mask, requester, request/grant`，并保留首访问延迟、返回时间及反压。CACTI 接收满足这些请求的物理宏组织；能量按实际服务的完整 word 事务计算。两个访问落在同一可广播 word 与落在两个不同地址，端口需求不同。不能将有效字节总量任意合并成不存在的物理事务。

尤其要分别记录 **计算端请求、SRAM 已服务请求和 stall**。增加算力后，原 SRAM 可能维持相同服务速率而使阵列停顿；如果只用"服务字节/已变长的执行时间"，会把饱和带宽误判为不需要增加 lane。只有为了保持目标 II 而新增的并行服务需求，才驱动 bank、字宽或端口升级。

Blackwell 的 operand collector 是官方公开的真实机制。PTX 允许连续 MMA 复用已加载的 A/B，但说明复用是机会性的，硬件仍可能重新加载。这证明实际 SRAM 请求不能由“数学上每个元素可以只读一次”唯一确定。具体合法的 collector 侧别还要匹配指令变体与目标架构。[PTX collector 语义]

已在本地工具中核验的 GB100 专用访问指标

本次通过 Nsight Compute 2025.4.1 的 `--chips gb100 --query-metrics` 核验了以下指标。当前可见 GPU 为 RTX 4090；这是 GB100 指标定义核验，未采集 B200 结果。

观察目标	GB100 指标（均已核对定义）	单位与边界
Tensor Core 从 SMEM 读 A	`l1tex__data_pipe_tc_wavefronts_mem_shared_op_utcmma_matrix_a.sum`	wavefront；Tensor Core 数据通路
Tensor Core 从 SMEM 读 B，1 CTA	`l1tex__data_pipe_tc_wavefronts_mem_shared_op_utcmma_matrix_b_scope_1cta.sum`	wavefront；2 CTA 有独立的 scope_2cta 指标
MMA 从 TMEM 读 C	`sm__mem_tensor_reads_op_utcmma_matrix_c.sum`	Byte
MMA 写 TMEM	`sm__mem_tensor_writes_op_utcmma.sum`	Byte；包含 D 写和 A shift，隔离实验需关闭 shift

计数器累计值能约束实际访问量和区间服务速率，不能唯一还原每拍的 bank read-enable。A/B 的 wavefront 也不能未校准便当成固定的有效字节数。普通 LSU 的 shared-load 计数不能替代上表的 Tensor Core 专用路径。完整的逐拍端口轨迹仍需要可观察的 RTL、足够细粒度的厂商模型或其它更直接证据。

昇腾公开 MMAD 的分形、C 初值来源和累加控制，但不公开全部内部端口时序。应比较初始化/读旧 C、同 A 换 B、同 B 换 A、不同 K 分块等真实请求变化，使用 MemoryL0 数据和可获得的仿真访存通路信息校准；逻辑初始化为零本身并不证明物理 SRAM 读使能一定关闭。

现有 `cost_model/dot_product_sram.py` 由固定结构生成事件，再按 word 量化；它没有实现厂商的 collector、内部装载器和逐拍端口仲裁。其“物理事务”是所声明候选布局下的模型事务，不是 B200/昇腾实测访存。当前结果可用于该抽象设计的成本比较，不能当作商用阵列的 SRAM lane 复原。

核验结果：microarchitecture_access_audit.json；原始 GB100 指标定义与选中指标 suffix 查询。下一步实测应固定合法指令与布局，分别扫 M/N/K、collector 配置和累加模式，同时记录访问量、活动周期与 stall。

---

9. 建议如何改造当前项目

第一阶段先选定一套 WS 契约：点积 PE 的 R/C/L、II 与流水线延迟、B 的预装及保持范围、P 的转发路径、外部 ACC 的实现和更新周期、阵列是否可按 M 分区。OS 使用单独的映射契约进行对照。每个精度要记录有效硬件形状，不能只记录一个峰值倍率。

优先级 / 模块	改造内容	验收方法
P0：领域与接口	增加 PrecisionMode、PEMicroarchitecture、DataflowMapping、AccumulatorPolicy、MemoryLayout；区分 K_alg / L_dot / K_tile / K_instruction 与 C/D alias	同一 GEMM 不同映射 MAC 覆盖恰好一次；流量守恒；声明所有驻留层次
P1：dot_product_sram.py 前端	用独立 OS/WS 事件生成器替代固定 M/N 广播与二选一 C 策略；显式 preload/stream/fold/drain；加入分区与尾块	标量 L=1 对齐经审核的 SCALE-Sim trace；点积小矩阵对齐整数参考与 RTL 请求轨迹
P1：C/D 与布局	独立生成初值读取、ACC RMW、最终输出；加入 bank 地址映射、合法合并、冲突、转发与 accumulator 依赖	β=0/1、F=1/多 fold、alias/独立 D、FIFO 极限、同 bank 同地址 hazard
P2：local_sram.py / cacti.py	复用宏成本适配器，改为消费实际服务事务；固定容量搜索 lane/word/port；服务 stall 反馈回计算调度	更窄带宽导致可解释的 stall；预装/排空期限；相同 trace 下能量可逐项复算
P2：matmul.py / SCALE-Sim 桥接	返回计算/供数/ACC/输出时间与 trace，而非仅周期÷mac_per_clock；缓存键纳入数据流、dot 长度、精度、布局和累加策略	相同峰值但不同扩展方向得到不同瓶颈；版本/源码 SHA 可追踪
P3：物理与芯片校准	用点积 RTL、SRAM compiler/综合结果校准 PPA；用实机 kernel 独立校准暴露接口的吞吐、复用和依赖	分别扫 M/N/K、精度、ACC 更新频次、B 重用、混合 load/store；保留计数器与误差分布

H100/B200/B300 或昇腾实测能够验证所选 kernel 与公开接口的性能，但不能仅由总延迟唯一反推内部 SRAM lane 数。应同时使用多个 shape、依赖链与带宽压力实验，区分阵列利用率、预装瓶颈和 ACC 服务瓶颈。对自定义 1:2:6/1:2:8 点积设计，RTL/综合的接口波形是更直接的校准依据。

> 建议用于比较的最小设计点不是"配比 + SRAM 带宽"，而是：
> **精度 → 有效 R/C/L 或分区数 → OS/WS 映射 → 驻留与累加协议 → 实际读写事件 → 同容量存储组织 → 延迟、面积与能量。**

---

10. 可复核证据与复跑边界

独立推导程序同时包含若干小整数 GEMM，枚举 OS/WS/IS 的每个 (m,n,k) MAC，检查没有遗漏和重复，并与直接矩阵乘法比较。它验证映射与数值覆盖，不验证浮点舍入或硬件时序。大矩阵表格采用在明确假设下推导的有效字节数，没有调用旧 CACTI 前端制造新数据流结果。

```
.venv/bin/python -B docs/reports/dot-product-dataflow-2026-10-29/evidence/probe.py
.venv/bin/python -B docs/reports/dot-product-dataflow-2026-10-29/evidence/derive_traffic.py
.venv/bin/python -B docs/reports/dot-product-dataflow-2026-10-29/evidence/residency_deadline.py
```

- SCALE-Sim 真实 demand 证据：包版本、生成器 SHA、逐行请求与库计数差异。
- 点积数据流独立推导与复跑脚本：输入假设、OS/WS/IS 流量、扩展方向和 MAC 覆盖。
- 当前固定流量前端：`generate_traffic` 是本报告指出的待改造位置；matmul.py 的 `simulate_systolic_array_cycle_count` 是现有 SCALE-Sim 消费路径。
- 运行文档保留旧实验，其面积/功率数值只对应旧的固定流量与宏组织假设，不能作为本报告新 WS 场景的预测。

LLMCompass 基础提交：`2e015fd2ee750e6cad8d7152df52551e5b41ef20`；工作区包含此前新增模型与文档。本地 SCALE-Sim wheel 没有可信 Git commit 元数据，因此报告记录版本及文件 SHA，不虚构安装包 commit。v3 的在线源码与公开论文用于接口调研，未在本次运行 v3 全系统仿真。

新增的预装/排空窗口验证与结果检查了 6 组 A 预装窗口、3 组 OS 排空窗口及持续输出积压。它采用明确的整数服务窗口与有限队列，不模拟具体 SRAM 的 bank/RAW 约束；用于证明“多周期写出”仍受可持续带宽约束。

主要一手来源

1. Gemmini 官方仓库：Architecture、Systolic Array and Transposer、Re-use Previous Preloads。
2. MFSA v1：§II-A 的多格式点积 WS 数据流。2026-10-14 预印本，不把其 PPA 外推到用户设计。
3. In-Datacenter Performance Analysis of a Tensor Processing Unit：ISCA 2017，§2 的阵列边界与外部 accumulator。
4. SCALE-Sim v3：ISPASS 2025 论文版本，§VII 的能量组件与 action counts；官方代码。
5. PTX ISA 官方文档及上文链接的 CUTLASS 指令指南：操作数位置与公开语义，不作为私有 PE 布局证明。
6. CACTI 官方实现：物理宏成本后端，不负责数据流映射。
